STAFF ENGINEER / ARCHITECT ASSIGNMENT · TECHNICAL ARCHITECTURE

PulseCrypto

A real-time cryptocurrency market viewer: a NestJS gateway that normalises Binance public streams into latest-state snapshots, and an Expo React Native client that renders those snapshots without owning transport.

System	**Nx monorepo · NestJS 11 · Expo 56**
Upstream	**Binance combined stream (ticker + partial depth)**
Downstream	**Native WebSocket `/market` + REST `GET /pairs/meta`**
Pairs	**BTCUSDT, ETHUSDT, SOLUSDT, DOGEUSDT, XRPUSDT**
Cadence	**Configurable snapshot interval, default 100 ms**
Document	**Architecture, protocol, memory bounds, ADRs**

Abstract. PulseCrypto is a market *viewer*, not an execution venue and not an event-replay system. The architecture is therefore organised around latest-state semantics: the backend retains one in-memory `MarketState` per pair, discards intermediate Binance events, and broadcasts a cloned snapshot at a configurable interval. The same bound is applied on egress: each mobile client has at most one in-flight send and one pending payload. This document describes the seams, algorithms, protocol, mobile state model, failure modes, and the explicit decisions not to introduce a broker, cache, or database.

Contents

1. Problem and constraints

The assignment is a staff-engineer practical: deliver a working viewer, and defend the architecture. Binance public streams can emit depth updates at 100 ms and ticker updates at 1 s, for five symbols concurrently. A naïve design that forwards every upstream frame to React Native would multiply CPU, allocations, battery, and network without improving what the user sees — the current price and a shallow book.

Constraints taken as first-class:

- **Latest-value semantics.** Intermediate states may be discarded. The product is not a matching engine and not a tape.
- **Bounded memory.** No unbounded per-client queues. A slow phone must not degrade the process.
- **Separation of concerns.** Binance payload shapes never reach the mobile app. UI never owns sockets.
- **Operational simplicity.** One backend process. No Kafka, Redis, RabbitMQ, or database unless a requirement appears that those tools uniquely solve.
- **Configurable cadence.** `MARKET_UPDATE_INTERVAL_MS` (default 100). `ORDER_BOOK_DEPTH` (5 | 10 | 20, default 10).

> The central thesis: *users care about now*. Everything expensive in a real-time mobile viewer is work spent on ticks nobody will render. The architecture is a machine for dropping that work at two choke points — the processor map, and the per-client pending slot.

2. System context and repository layout

The workspace is an Nx 23 integrated monorepo with pnpm 11 and Node 22 LTS. TypeScript 6 resolves internal packages through `exports` and `customConditions: ["@pulse-crypto/source"]`, not deprecated `baseUrl`/`paths`.

PACKAGE	ROLE	MAY DEPEND ON
`apps/api`	NestJS REST + native WS gateway, Binance feed, processor	contracts, market-domain, shared
`apps/mobile`	Expo RN client	contracts, shared — *not* market-domain
`libs/contracts`	TradingPair, MarketState, WS/REST DTOs, discriminators	shared only
`libs/market-domain`	Book bound, spread, pressure, feed update types	contracts, shared
`libs/shared`	Bounded reconnect delay sequence	—

Mobile is forbidden from importing `market-domain` at the ESLint module-boundary layer. Derived metrics are computed once on the server and shipped in the snapshot. That keeps the phone a renderer of an application protocol, not a second copy of exchange maths.

Feed, processor, and gateway live in one Nest module (`MarketModule`) so the feed can inject the processor as `MARKET_FEED_LISTENER` and the processor can inject the gateway as `MARKET_SNAPSHOT_SINK` without circular modules. The extra tokens are the inversion: both sides remain unit-testable with fakes.

3. Logical architecture

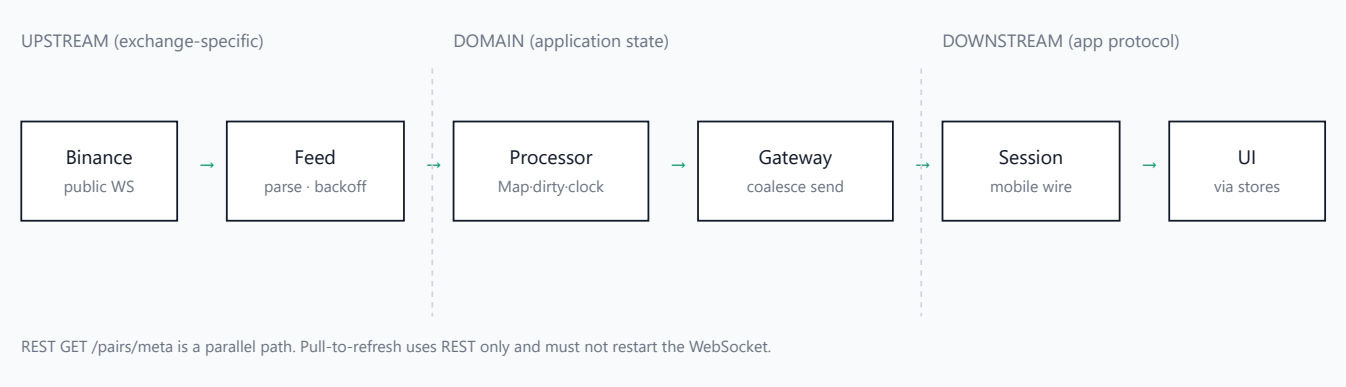

Figure 1. Runtime pipeline. Vertical dashed lines are trust/knowledge boundaries: Binance JSON stops at the feed; the UI never sees sockets.

LAYER	OWNS	MUST NOT KNOW
BinanceFeedService	Combined-stream URL, socket, parse, single reconnect loop	Mobile clients, Nest WS adapter
MarketProcessor	Per-pair accumulator, dirty flag, interval, domain maths	WebSocket send, Binance field names after parse
MarketGateway	ClientSession map, last encoded snapshot, handshake	Binance stream names
MarketSession	Mobile WS + REST lifecycle	React tree
MarketStore / FavoritesStore	Latest markets vs persisted symbols	Transport
Screens	Render, search, navigation state	Sockets, Binance

3.1 Product surfaces

The following frames are from the running Expo client against the live gateway (Binance public feed). They are the assignment's two screens and the failure path — not a separate slide deck.

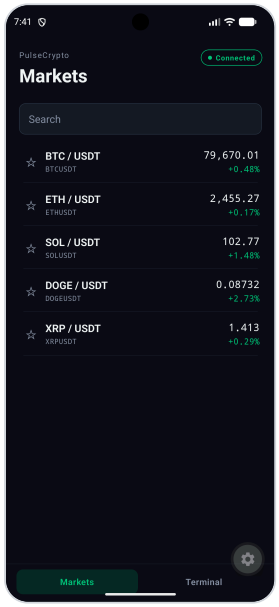

Figure 2. Watchlist: five pairs, live price and 24h %, connection pill Connected, search does not mutate market state.

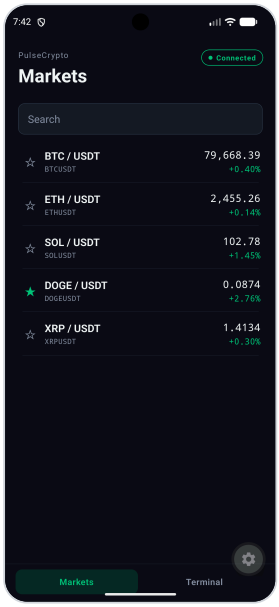

Figure 3. Favourites are a separate store. DOGE is starred; that preference is AsyncStorage, not a market tick.

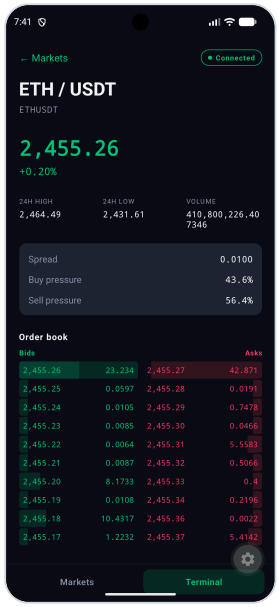

Figure 4. Details (ETH): last price, 24h high/low/volume, spread, buy/sell pressure, bounded bids/asks.

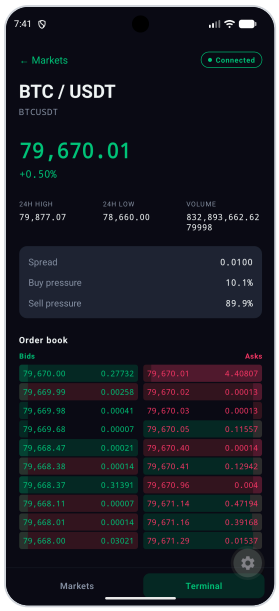

Figure 5. Details (BTC): same protocol payload, one-pair Zustand selector so other rows do not re-render.

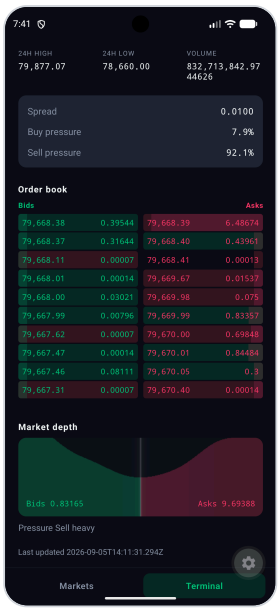

Figure 6. Bounded book plus depth visualisation. Last-updated timestamp is the snapshot's `updatedAt`, not a REST poll.

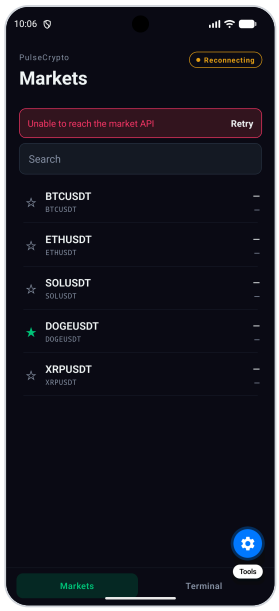

Figure 7. Failure path: Reconnecting pill, REST error + Retry, pair list retained. Live ticks resume when the socket returns a snapshot.

Figure 7 is a cold or failed session (prices show as placeholders until a snapshot arrives). When a snapshot has already been applied, disconnect changes the pill and shows a stale hint but does not clear the map — that is the store invariant, covered in §11 and mobile tests.

---

4. Upstream ingestion

The feed subscribes to a Binance *combined* stream, not one socket per pair. For each of the five symbols it requests two streams:

```
wss://stream.binance.com:9443/stream?streams=
  btcusdt@ticker / btcusdt@depth10@100ms / … (×5 pairs)
```

`@ticker` supplies last price, 24h percent change `P`, high, low, and quote volume. Mini-ticker is rejected because it omits percent change, which the watchlist requires. `@depthN@100ms` is Binance *partial book depth*: each event is a replacement snapshot of the top N bids and asks, not a diff. N is taken from `ORDER_BOOK_DEPTH`.

This is a deliberate rejection of a local order-book engine. Diff-depth (`@depth@100ms`) plus REST snapshot sync would require sequence numbers, gap detection, and re-seeding. The UI only needs ten levels. The exchange already publishes that snapshot. Accuracy claim (ADR-013): the book is the exchange's top N at the stream's 100 ms cadence, not a sequential reconstruction of the full book. Levels beyond N are omitted. Gaps during reconnect are healed by the next snapshot.

Parsing is defensive. Malformed frames are dropped and logged; they must not tear down the socket or the processor. The feed talks to a `MarketFeedListener` (`onTicker` / `onOrderBook`) so tests inject a spy instead of Binance.

Reconnect: a single timer, delays `[1s, 2s, 4s, 8s, 16s, 30s]` via `reconnectDelayMs` in `@pulse-crypto/shared`. Success resets the attempt counter. Module destroy detaches the socket and clears the timer. Two concurrent reconnect loops are a defect, not a feature.

---

5. Market processor

Conceptually the processor is:

```
states: Map<TradingPair, PairAccumulator>
dirty: boolean
timer: setInterval(intervalMs) → publishIfDirty()
```

Ticker and depth handlers mutate the accumulator in place and set `dirty = true`. They do not enqueue events. There is no per-tick array that can grow with upstream rate.

Merge and completeness. Depth can arrive before ticker. The accumulator retains the book, but `toMarketState` returns `undefined` until `lastPrice`, 24h change, high, low, and volume are all present. Incomplete pairs are omitted from the published array. Spread and pressure remain `null` until both sides of the bounded book are usable; the snapshot still ships so the watchlist can show price without inventing a spread.

Snapshot clock. On each interval, if `dirty` is false the tick is skipped — idle state is not rebroadcast. If dirty, the flag is cleared, complete pairs are cloned (book levels copied), sorted by pair, and handed to `MarketSnapshotSink.publish`. Cloning is required so later mutations cannot alias into an in-flight JSON encode.

Default interval 100 ms ⇒ maximum ~10 snapshots/s to mobile, independent of how many Binance frames arrived in that window. Ticker (1 s) and depth (100 ms) are thereby collapsed to one downstream cadence.

Complexity: O(P) per publish with P = 5, plus O(N log N) on each depth event to bound and sort N ≤ 20 levels. Memory is O(P × N) and does not grow with time or with client count (clients are the gateway's problem).

---

6. Domain calculations

Implemented in `@pulse-crypto/market-domain`, tested without Nest or sockets.

Bound. Drop non-finite or non-positive price/quantity. Sort bids descending, asks ascending. Slice to depth. Copy levels so callers cannot mutate the retained book.

Spread. `bestAsk − bestBid`. `null` if either side is missing or non-finite. No silent zero that would look like a locked market.

Pressure (share of displayed depth, not of the full exchange):

```
bidVolume = Σ bid.quantity
askVolume = Σ ask.quantity
buyPressure  = bidVolume / (bidVolume + askVolume) × 100
sellPressure = 100 − buyPressure
```

If total volume is 0, both pressures are `null` rather than NaN. This is a *visible-book* heuristic, documented as such. It is not toxicity, not imbalance of the full book, and not a trading signal.

7. Gateway and slow consumers

Transport is Nest native WebSockets (`@nestjs/platform-ws`), path `/market`, same HTTP port as REST. Socket.IO is rejected (ADR-012): the client uses the platform `WebSocket` API and a documented JSON protocol; rooms and engine.io fallbacks are unused cost.

On connect the gateway:

1. Wraps the socket in `ClientSession`.
2. Sends `connection.ready` immediately (`sendNow`).
3. If a snapshot has already been published, `offer`s the last encoded payload so a late joiner does not wait a full interval.

On `publish(snapshot)` the gateway JSON.stringifies once, stores `latestEncoded`, and offers that same string to every session. Encoding is not repeated per client.

Coalescing algorithm (`ClientSession`):

```
pending?: string
sending: boolean

offer(payload):
  pending = payload          // overwrite, never push
  flush()

flush():
  if sending or pending is empty or socket not OPEN: return
  take pending, sending = true
  socket.send(payload, cb):
    sending = false
    if error: drop client
    else flush()             // send the newest overwrite, if any
```

Invariants: at most one in-flight send; at most one pending snapshot; newer snapshots replace pending. Memory per client is O(1) payloads, not O(backlog). A client that cannot drain 10 Hz still converges to the latest state rather than a delayed queue of stale books. That is the assignment's slow-consumer requirement, applied on egress as well as ingress.

Disconnect and socket error remove the session and call `dispose` (clears pending, ignores further offers). No send after close.

```
t=0 snapshot A → send A (in-flight)

t=1 snapshot B → pending = B

t=2 snapshot C → pending = C (B discarded)

t=3 A completes → send C
```

Figure 2. Slow consumer: B is dropped; C is the only state the client will ever see after A.

8. Wire protocol and REST

Mobile depends on this protocol, never on Binance frames. Messages are JSON objects with a `type` discriminator.

TYPE	WHEN	PAYLOAD
`connection.ready`	After accept	timestamp only
`market.snapshot`	~interval, if dirty and ≥1 complete pair	`data: MarketState[]`
`error`	Server-side failure that is safe to surface	`code`, `message` — no stacks, no Binance strings

`MarketState` includes pair, lastPrice, change24hPercent, high/low/volume 24h, spread, buyPressure, sellPressure, bids[], asks[], updatedAt. Nullable derived fields are `null`, not omitted, so clients can distinguish "not yet calculable" from a missing key.

REST `GET /pairs/meta` returns display metadata (symbol, displayName, tradingStatus, 24h high/low/volume). The assignment allows a static provider; the API is still behind an interface so a live exchange metadata source could replace it. Errors use `{ error: { code, message } }`.

Lifecycles are independent. Pull-to-refresh calls REST only. It must not close or recreate the WebSocket. REST failure keeps previous pair metadata and shows Retry. WS failure keeps previous markets and shows a stale hint.

9. Mobile architecture

```
Screens  →  Zustand stores  →  MarketSession  →  WebSocketService / REST
               ↑ writes                          ↑ owns sockets
```

`MarketSession` is the composition root. It hydrates favourites, opens the socket, fetches metadata, and maps protocol messages into store actions. Screens never construct a `WebSocket`.

MarketStore holds `markets: Record<TradingPair, MarketState>`, connection status (`connecting | connected | reconnecting | disconnected | error`), pair metadata, and surfaceable errors. Applying a snapshot replaces latest state per pair; disconnect changes status and *must not* `{}` the map (offline requirement).

FavoritesStore holds an array of symbols. Persistence is isolated behind `FavoritesStorage` (AsyncStorage in production, in-memory in tests). Only symbols are serialised. Live ticks never touch disk.

Navigation is local `selectedPair` in `App`. Two screens do not justify React Navigation.

Render isolation. Watchlist rows and details subscribe with `selectMarket(pair)`. The list screen subscribes to metadata, search, and errors — not the whole `markets` map. A BTC tick should not re-render ETH. Tick flash is a 350 ms background on the component that already subscribed; order-book bars are width plus a quantity flash, not a new `Animated.Value` per level per snapshot (that would allocate on the 10 Hz path).

Search filters the supported-pair list in memory. It does not write market state. Five pairs do not require FlatList virtualisation; that is documented as a future trigger, not premature machinery.

10. Performance model

STAGE	BOUND	WHAT WOULD BREAK IT
Upstream	10 streams (5 ticker + 5 depth)	Per-event logging; full-book diffs
Processor	O(P·N) memory, 10 Hz publish max	Append-only queues; publishing when !dirty
Encode	One JSON string per publish	Per-client stringify
Gateway	2 payloads / client	Queue of snapshots
RN store	Replace by pair key	Subscribing the list to `markets`
RN render	One row / pair / tick	Context that wraps the tree; memo everywhere "just in case"

Philosophy: measure or reason, then apply a targeted bound. Memoisation as decoration is treated as a smell. If pair count moved into the hundreds, the next levers are virtualised lists, cheaper book diffs, server-side subscription filters, and possibly binary frames — not a rewrite of latest-state.

11. Resilience and errors

FAILURE	SYSTEM BEHAVIOUR	USER-VISIBLE
Binance disconnect	Feed backoff; processor keeps last accumulators; gateway still serves last snapshot to connected phones	Phones stay "connected" to *us* until our socket dies; prices freeze until upstream returns
API process down	Mobile WS closes; single reconnect loop 1s→30s	Pill reconnecting/disconnected; "Showing last received prices"; numbers remain
REST /pairs/meta fails	Previous metadata kept; refresh flag cleared; WS untouched	Error banner + Retry
Malformed Binance JSON	Drop frame, log once-class of error, continue	None
Malformed client frame	Must not crash the gateway	Optional protocol error message
App process kill	Favourites hydrate from AsyncStorage; markets refill after snapshot	Stars persist; prices arrive after connect

Logging is lifecycle and error only. Per-tick logs are forbidden: at 10 Hz they become a performance bug and a useless firehose.

Figure 7 shows the reconnecting pill together with a REST failure banner and Retry. Pair rows remain. If a snapshot had already been applied, those prices would stay; this capture is a session that had not yet received ticks, so the price column is empty until the next `market.snapshot`.

Timers, socket listeners, and reconnect handles are cleared on Nest module destroy and on mobile session stop. Leaking a second reconnect loop is an explicit test concern.

12. Testing and CI

Tests target behaviour and failure modes, not coverage percentages. Binance and the public internet are not in the unit loop.

- **Processor:** replace vs insert, multi-pair isolation, interval skip when clean, configurable interval, spread/pressure, zero volume, depth clamp, completeness gate (book without ticker is unpublished).
- **Feed:** URL construction, parse, invalid messages, disconnect, backoff sequence, cleanup. Socket factory is injected.
- **Gateway:** connect handshake, broadcast, multi-client, coalesce under backpressure, disconnect cleanup.
- **REST:** shape of `/pairs/meta`, error envelope.
- **Mobile:** store apply, disconnect preserves markets, favourite persist/restore, search, watchlist/details render, session does not clear on REST failure.

CI (GitHub Actions): `pnpm nx run-many -t lint,typecheck,test` then build `api`, `contracts`, `shared`, `market-domain`. Expo native `build` is omitted: it needs EAS credentials and is not what the architecture tests.

Live Binance is a demonstration (recording / viva), not a gate.

13. Architectural decisions

ADR	DECISION	TRADE-OFF
001–002	Latest-state + 100 ms clock	Intermediate ticks discarded
003, 008, 009	In-memory only; no DB; no broker	State dies with the process; no independent scale-out of ingest vs fan-out
004, 013	Bounded book from partial depth	Not a full-book reconstruction
005	Zustand, two stores	Less ceremony than Redux; discipline required on selectors
006	AsyncStorage for favourites only	Device-local preferences
007	REST metadata / WS markets	Two clients to configure; two failure domains (intentional)
012	Native WS, not Socket.IO	No rooms, acks, or HTTP long-poll fallback
014	Full ticker, not mini-ticker	Heavier ticker payload; required 24h %
015	Snapshot sink interface	One extra token for testability
016	Per-client coalescing	Slow clients skip snapshots; they never skip the *latest*
010–011	Nx 23, Node 22	Contributors cannot stay on Node 18

13.1 What was refused

Kafka, Redis, CQRS, event sourcing, and microservices do not pay for themselves on five pairs and one process. They would move the assignment's complexity into operations without changing the user-visible property (current price on a phone). Socket.IO would add a second protocol. A local matching engine would add snapshot-sync bugs. Redux and React Navigation would add ceremony for two screens and two stores.

13.2 Scaling horizon

This design is sized to five pairs, one process, and a handful of mobile clients. Honest scaling:

- **Hundreds of pairs:** virtualised watchlist, server-side subscription filters so a client does not receive unused books, cheaper book diffs on the wire.
- **Many concurrent clients:** split ingest from fan-out; share the latest-state map; consider binary frames; still coalesce per client.
- **Multiple API instances:** in-memory state is no longer sufficient; a shared latest-state store or sticky ingest node appears. That is the first moment Redis (or equivalent) earns a place — as a *latest-value* store, not as an event log.
- **Execution / audit:** stop dropping ticks. Introduce sequencing, persistence, and a real book. That is a different product.

13.3 AI-assisted implementation

Cursor was used for scaffolding, tests, and review. Architecture and ADRs were specified before generation. AI output was treated as untrusted until typecheck, tests, and the failure-path behaviour (disconnect without wipe) were verified. That workflow is documented in `docs/AI_DEVELOPMENT.md`.

References (in-repo)

`docs/ARCHITECTURE.md` · `docs/DECISIONS.md` · `docs/REALTIME_PROCESSING.md` · `docs/WEBSOCKET_PROTOCOL.md` · `docs/API_CONTRACT.md` · `docs/MOBILE_ARCHITECTURE.md` · `docs/PERFORMANCE.md` · `docs/TESTING.md` · `docs/ERROR_HANDLING.md`